

The Pigeonhole Principle is one of the most intuitive theorems in discrete mathematics because if you place $n+1$ objects into n boxes, at least one box must contain more than one object. Despite its simplicity, this principle underlies surprisingly deep results across combinatorics, number theory, and computer science. For this project, I chose to formalize it in Lean 4 using Mathlib, the community mathematical library for Lean. My goal was to understand what it actually means to translate a familiar informal proof into a machine-verified formal one, and to experience firsthand where the difficulties arise in that translation.

Informally, the Pigeonhole Principle is a statement about counting. Formally, it becomes a statement about functions: there is no injective function from a set of size $n+1$ into a set of size n . Recall that a function is injective (or one-to-one) when distinct inputs always produce distinct outputs. If such an injection existed, it would be a way of embedding $n+1$ distinct items into n slots without collision, which is impossible.

In Lean, finite sets of a fixed size k are represented by the type `Fin k`, whose elements are the natural numbers 0 through $k-1$. Injectivity is provided by Mathlib as `Function.Injective`. So the formal statement becomes:

```
theorem pigeonhole (n : ℕ) :  
  ¬ ∃ f : Fin (n + 1) → Fin n, Function.Injective f
```

This matches the informal statement precisely, which is one of the satisfying aspects of working in Lean: the gap between what you write down mathematically and what you write in code can be quite small when the right library definitions are in place.

The formal proof follows a clean three-step argument by contradiction. First, assume such an injection f exists. Second, apply the Mathlib theorem `Fintype.card_le_of_injective`, which states that any injection between finite types forces the cardinality of the domain to be at most the cardinality of the codomain. Third, use the fact that `card(Fin k) = k` (provided by `Fintype.card_fin`) to

rewrite both cardinalities as plain natural numbers, yielding the inequality $n+1 \leq n$. Since this is false for all natural numbers, we have our contradiction.

The tactic `omega` closes the final arithmetic step automatically. This is a decision procedure built into Lean for linear arithmetic over integers and natural numbers, and it handles goals like " $n+1 \leq n$ implies False" without any further manual work.

Working through this project illustrated several things that a traditional mathematics course does not make explicit. The most significant is how much work the informal phrase "clearly" or "it follows that" actually conceals. In a standard proof, one might write: "since f is injective, the domain cannot be larger than the codomain." In Lean, this step requires knowing the exact name of the relevant library theorem, understanding its type signature, and ensuring that Lean's unifier can match the goal with the lemma's conclusion. None of this is difficult once you know the library, but finding the right lemma requires searching Mathlib systematically.

A second observation is the role of types. In informal mathematics, the statement " $\text{Fin } n$ is a finite set of size n " is obvious from the definition. In Lean, the connection between a type and its cardinality is mediated by the `Fintype` typeclass, which must be synthesized automatically. The fact that Lean can do this synthesis transparently for `Fin n` is a non-trivial feature of Mathlib's design, and it was helpful to understand why it works rather than just accepting that it does.

Finally, this project reinforced the value of naming intermediate results clearly. My final file separates the proof into helper lemmas (`card_fin_eq` and `injective_card_le`) before stating the main theorem. This makes each logical step visible and checkable individually, which is both good proof style and useful pedagogically. The file also includes an alternate one-line proof using `simp` to show that the same result can be obtained more concisely once the pieces are familiar.

Formalizing the Pigeonhole Principle turned out to be an ideal entry point into Lean for someone with a discrete mathematics background. The mathematical content is elementary enough that I could focus on the mechanics of formalization, and the

theorem touches on concepts — injectivity, finite sets, cardinality — that appear throughout the Mathlib library. The most important takeaway is that formal proof is not just a translation of informal proof; it is a discipline that forces you to be explicit about every step, every definition, and every dependency. That discipline, once practiced even at a small scale, changes how you read and write informal mathematics as well.